System zapisów na zajęcia

Etap 2: architektura, prototypy i struktury danych

Kamil Kręcigłowa Bartosz Królik Krzysztof Krówka

21 października 2026

Inżynieria Oprogramowania, zima 2026/2027

Temat i plan prezentacji

Temat 2: System zapisu na zajęcia

- ograniczona liczba miejsc, różne rozmiary grup
- katalog: kursy, grupy, prowadzący, harmonogram
- wymagania wstępne (zaliczone przedmioty)
- wykrywanie kolizji w planie
- listy oczekujących z automatycznym zapisem

Plan

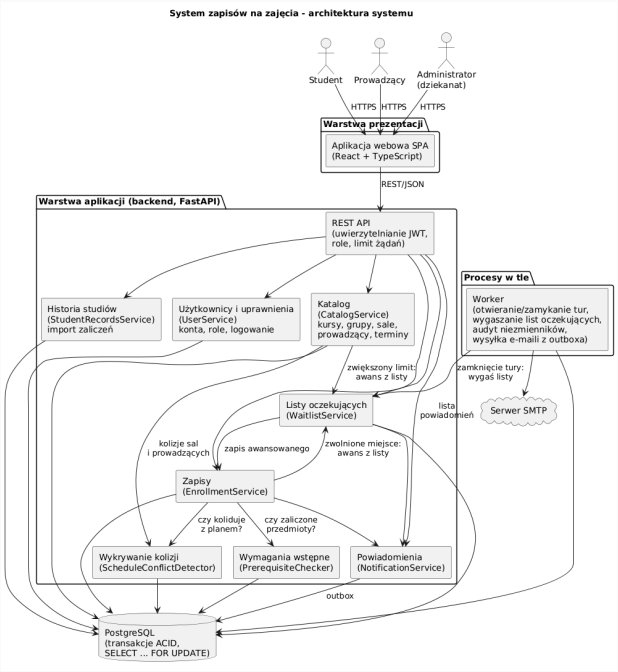
1. Architektura i stos technologiczny
2. Przypadki użycia
3. Struktury danych (ER)
4. Prototypy obszarów ryzyka
5. Mock-up interfejsu

Pokrycie wymagań

Wymaganie	Przypadki	Tabele (ER)	Prototyp
ograniczona liczba miejsc	UC2, UC13	`section.capacity`, `enrolled_count`, `waitlist_limit`	scen. 1, 5
katalog i harmonogram	UC1, UC11, UC12	`course`, `section`, `lecturer`, `meeting`, `room`, `term`, `enrollment_round`	scen. 6
różne rozmiary grup	UC12, UC13	`section.capacity`, `section.kind`, `room.seats`	scen. 1, 4, 5
wymagania wstępne	UC3, UC11, UC15	`course_prerequisite`, `completed_course`	scen. 2, 6
kolizje w harmonogramie	UC4, UC12	`meeting` (dzień, godziny, parzystość)	scen. 3
listy oczekujących	UC5, UC7, UC9, UC18	`waitlist_entry`, `notification`	scen. 4, 5, 6

Architektura

System zapisów na zajęcia - architektura systemu



Architektura: moduły i odpowiedzialności

- **SPA** (React + TS) dla trzech ról
- **REST API** (FastAPI): JWT, role, limit żądań
- `EnrollmentService` – *jedyny* moduł zmieniający liczbę zajętych miejsc
- `PrerequisiteChecker`, `ScheduleConflictDetector` – reguły zapisu
- `WaitlistService` – kolejka i automatyczny awans
- `NotificationService` – outbox w bazie
- **Worker**: otwiera i zamyka tury, wygasza listy, audytuje niezmienniki, wysyła e-maile
- **PostgreSQL** – jedno źródło prawdy

Główne wyzwanie

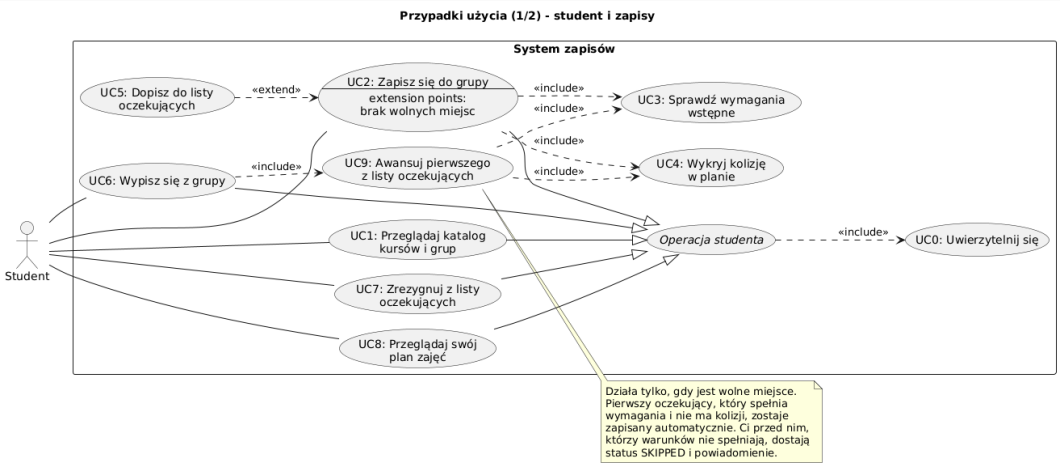
Otwarcie tury: setki studentów naraz walczą o te same miejsca. Zapis do grupy serializuje blokada jej wiersza, a zapisy studenta – blokada wiersza studenta.

Stos technologiczny

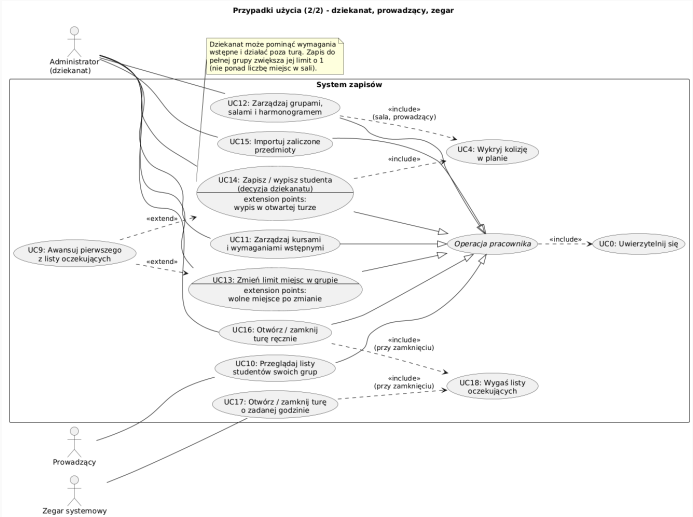
Element	Technologia	Uzasadnienie
Frontend	React + TypeScript	typowanie, łatwy katalog, formularze, siatka planu
Backend	Python 3.12, FastAPI, SQLAlchemy	szybkie prototypowanie, OpenAPI, walidacja (Pydantic), async
Baza danych	PostgreSQL 16	ACID, blokady wierszy, indeksy częściowe, `TIME`, `JSONB`
Zadania w tle	własny worker (APScheduler)	outbox i tury w bazie – broker zbędny
Uwierzytelnianie	JWT, docelowo SSO uczelni	bezstanowe API, proste role
Wdrożenie	Docker Compose	to samo środowisko dla całego zespołu

Przypadki użycia

Przypadki użycia (1/2): student



Przypadki użycia (2/2): dziekanat, prowadzący, zegar



UC2: Zapisz się do grupy

Przebieg główny

1. Student wybiera grupę z katalogu (UC1).
2. Blokada wiersza **studenta**, potem **grupy**.
3. Wymagania wstępne (UC3).
4. Kolizje z planem (UC4).
5. Wolne miejsce $\Rightarrow$ zapis, licznik $+1$; wpisy na listach innych grup tego przedmiotu i typu $\Rightarrow$ CANCELLED.
6. Grupa pojawia się w planie (UC8).

Scenariusze alternatywne (odmowa z przyczyną)

- brak zaliczenia / kolizja (który przedmiot, która grupa)
- już ma grupę tego przedmiotu i typu (wykład + lab – wolno)
- już zapisany lub już czeka na tę grupę
- tura zamknięta
- brak miejsc $\Rightarrow$ **UC5** (<<extend>>): koniec listy, o ile nie przekroczono `waitlist_limit`

UC9: Awans z listy oczekujących

Wywoływany w **tej samej transakcji**, która zwolniła miejsce: wypis (UC6), zmiana limitu (UC13), decyzja dziekanatu (UC14).

1. Dopóki jest wolne miejsce – bierz pierwszego `WAITING`.
2. Sprawdź **ponownie** wymagania, kolizje i regułę „jedna grupa danego typu".
3. Spełnia $\Rightarrow$ zapis (`source = WAITLIST`) i powiadomienie.
4. Nie spełnia $\Rightarrow$ `SKIPPED` z powodem, powiadomienie, następny.

Decyzje projektowe

- bez wolnego miejsca nikogo nie pomijamy – student może jeszcze usunąć kolizję
- zamknięcie tury (UC16/UC17) $\Rightarrow$ UC18: listy wygasają (`EXPIRED`)
- dziekanat (UC14): może pominąć wymagania i działać poza turą; zapis do pełnej grupy = limit $+1$ (max. liczba miejsc w sali)

Niezmiennik

Nikt nie czeka na liście, gdy w grupie są wolne miejsca.

Struktury danych


app_user
PK id bigserial
UQ email varchar(255)
password_hash varchar(255)
full_name varchar(200)
role varchar(16)
created_at timestamptz
notification
PK id bigserial
FK user_id bigint
type varchar(32)
payload jsonb
created_at timestamptz
sent_at timestamptz
lecturer
PK FK user_id bigint
title varchar(50)
department varchar(100)
student
PK FK user_id bigint
UQ index_no varchar(10)
program varchar(100)
semester smallint
section_lecturer
PK FK section_id bigint
PK FK lecturer_id bigint
enrollment
PK id bigserial
FK student_id bigint
FK section_id bigint
status varchar(16)
source varchar(16)
enrolled_at timestamptz
dropped_at timestamptz
waitlist_entry
PK id bigserial
FK student_id bigint
FK section_id bigint
status varchar(16)
created_at timestamptz
resolved_at timestamptz
skip_reason varchar(255)
FK enrollment_id bigint
completed_course
PK FK student_id bigint
PK FK course_id bigint
PK FK term_id bigint
grade numeric(2,1)
passed boolean
section
PK id bigserial
FK course_id bigint
FK term_id bigint
code varchar(10)
kind varchar(16)
capacity smallint
enrolled_count smallint
waitlist_limit smallint
term
PK id bigserial
UQ code varchar(10)
starts_on date
ends_on date
course
PK id bigserial
UQ code varchar(20)
name varchar(200)
ects smallint
description text
meeting
PK id bigserial
FK section_id bigint
FK room_id bigint
weekday smallint
start_time time
end_time time
week_parity varchar(4)
room
PK id bigserial
building varchar(20)
number varchar(20)
seats smallint
enrollment_round
PK id bigserial
FK term_id bigint
name varchar(50)
opens_at timestamptz
closes_at timestamptz
status varchar(16)
course_prerequisite
PK FK course_id bigint
PK FK required_course_id bigint
PK klucz główny
FK klucz obcy (kolor = tabela docelowa)
UQ wartość unikalna
kursywa – NULL dozwolony

Najważniejsze decyzje w modelu danych

- **Kurs $\neq$ grupa**: `course` w katalogu, `section` w semestrze z własnym limitem i typem
- **Harmonogram** w `meeting`: dzień, godziny, parzystość tygodni semestru, sala
- **Licznik** `enrolled_count`: sprawdzenie limitu i zajęcie miejsca to jeden `UPDATE`
- **Historia zamiast kasowania**: `DROPPED`, `PROMOTED`, `SKIPPED`, `CANCELLED`, `EXPIRED`
- **Indeksy częściowe**: jeden aktywny zapis / wpis na liście
- **Kolejność listy** = `id` z sekwencji (nadawane pod blokadą grupy)
- **Tury** w `enrollment_round`, najwyżej jedna otwarta w semestrze
- `passed` wyliczane z oceny, trigger blokuje cykle wymagań
- **Outbox**: powiadomienie w tej samej transakcji co awans

Reguła kolizji

Dwa terminy kolidują, gdy:

- ten sam dzień tygodnia,
- godziny się przecinają (styk końcem *nie* koliduje),
- tygodnie się pokrywają: `ALL` – ze wszystkim, `ODD` – z `ODD`, `EVEN` – z `EVEN`,
- ten sam semestr.

```
mine.weekday = other.weekday
AND mine.start_time < other.end_time
AND other.start_time < mine.end_time
AND (mine.week_parity = 'ALL'
  OR other.week_parity = 'ALL'
  OR mine.week_parity = other.week_parity)
```

Plan: pon 10:00–11:30,
śr 12:00–13:30 (tyg. nieparzyste)

Nowa grupa	**Wynik**
pon 11:00–12:30	KOLIZJA
pon 11:30–13:00 (styk)	ok
wt 10:00–11:30	ok
śr 12:00–13:30 parzyste	ok
śr 13:00–14:30 co tydzień	KOLIZJA

Funkcja SQL
`schedule_conflicts(student, grupa)`.

Prototypy

Obszary ryzyka i prototyp

1. **Wyścig o ostatnie miejsca** (*check-then-act*)
2. **Wymagania wstępne**
3. **Kolizje** – styk końcem, tygodnie parzyste
4. **Automatyczny zapis z listy**: każde miejsce zajęte raz, kolejność, pomijanie niespełniających warunków
5. Jedna grupa danego typu, limit listy, ponowny zapis
6. Decyzje dziekanatu, zamknięcie tury

`prototyp/prototyp_zapisow.py`

- Python + SQLite, tylko biblioteka standardowa
- równoległe wątki z barierą
- `check_invariants()` po każdym scenariuszu

Schemat docelowy (`schemat.sql`) sprawdzony na **PostgreSQL 16**.

Scenariusz 1: 30 studentów, 10 miejsc

Wersja naiwna: odczyt licznika, potem osobny zapis

```
SELECT enrolled_count, capacity ...
-- okno wyscigu
INSERT INTO enrollment ...
UPDATE section SET enrolled_count = :n+1
```

30 zapisanych / 10 miejsc, licznik = 1

`CHECK (enrolled_count <= capacity)` tego *nie* wykrywa – licznik jest zaniżony. Wykrywa to widok `section_invariant_violations`.

Wersja bezpieczna: jedna atomowa operacja

```
UPDATE section
   SET enrolled_count = enrolled_count + 1
 WHERE id = :section
   AND enrolled_count < capacity;
-- 0 wierszy => lista oczekujacych
```

10 zapisanych, 20 na liście, niezmienniki OK

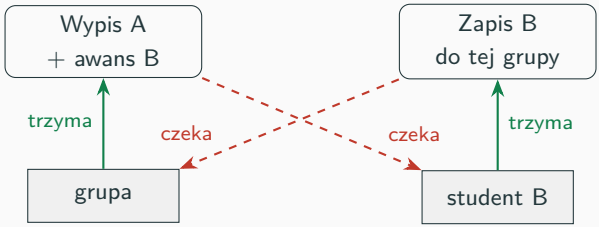
Wynik powtarzalny przy każdym uruchomieniu.

Scenariusze 4–6: lista oczekujących

- L01 pełna (2/2), czekają: 3, 4, 5, 6. Student 3 zapisuje się na kolidujące BD L05.
- Limit bez zmian $\Rightarrow$ nikt nie awansuje, student 3 nadal czeka.
- Student 1 się wypisuje $\Rightarrow$ 3: `SKIPPED` (kolizja z L05), 4: `PROMOTED`.
- Dwa wypisy w tej samej chwili $\Rightarrow$ awansują dokładnie dwaj pierwsi, kolejność zachowana.
- Piotr czeka na L03 i zapisuje się na L05 $\Rightarrow$ wpis na L03 `CANCELLED`, zwolnione miejsce na liście dostaje Kuba.
- Ponowny zapis do tej samej grupy $\Rightarrow$ odmowa („już zapisany” / „już czeka”).
- Dziekanat zapisuje Ewę bez wymagań do pełnej L07 $\Rightarrow$ limit $2 \to 3$, Kuba nadal czeka.
- Zamknięcie tury $\Rightarrow$ lista wygasa; wypis po turze przez dziekanat nikogo nie zapisuje.

Wszystkie niezmienniki spełnione po każdym kroku.

Blokady w PostgreSQL: zakleszczenie przy awansie



Sprawdzone na PostgreSQL 16 (dwie sesje):

`ERROR: deadlock detected`

Zapis: **student** → **grupa**.
Awans: **grupa** → **student B** – odwrotnie.

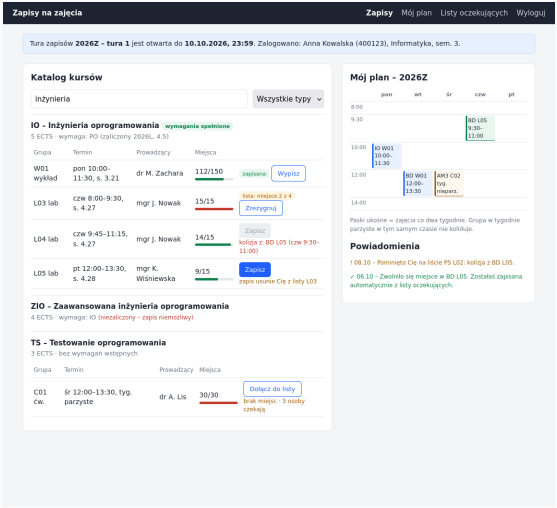
Rozwiązanie:

```
SELECT 1 FROM student
 WHERE user_id = :b
   FOR UPDATE NOWAIT;
```

Zajęty ⇒ błąd 55P03 od razu, `ROLLBACK` i ponowienie całej transakcji. B nie jest pomijany (nie traci kolejki). Tak samo obsługujemy 40P01 i 40001.

Mock-up

Mock-up: ekran zapisów studenta



- katalog z zajętością miejsc
- kurs zablokowany przez niezaliczony przedmiot
- „Zapisz" zablokowane przez kolizję, z podaniem grupy
- pozycja na liście, „Zrezygnuj", „Dołącz do listy"
- ostrzeżenie: zapis usunie z listy innej grupy
- plan tygodnia z zajęciami co dwa tygodnie
- powiadomienia o awansie i pominięciu

Podsumowanie

Podsumowanie i kolejny etap

Gotowe

- architektura i stos technologiczny
- przypadki użycia UC0–UC18 z głównymi scenariuszami
- diagram ER + `schemat.sql` (PostgreSQL 16)
- działający prototyp 6 scenariuszy ryzyka
- mock-up ekranu zapisów

Etap 3 (28.X)

- diagramy sekwencji: zapis, wypis z awansem, otwarcie tury
- diagram klas / komponentów z metodami
- uprawnienia ról i obsługa błędów
- test blokad i wydajności na PostgreSQL
- zamiana grupy bez utraty miejsca

Dziękujemy!

Pytania?